

Python Memory Model — সহজ বাংলা নোট

Variable, Object, Reassignment ও Mutation — ধাপে ধাপে ব্যাখ্যা

উৎস: CSC110/111 Course Notes (Toronto) · বিষয়: 6.4 The Python Memory Model

মূল ধারণা: Variable আসলে object নয়

সবচেয়ে গুরুত্বপূর্ণ পয়েন্ট হলো — Python-এ একটা variable নিজে কোনো data রাখে না। বরং variable একটা id রাখে, যেটা একটা object-কে নির্দেশ করে, আর সেই object-টাই আসল data ধরে রাখে।

```
x = 3
word = 'bonjour'
```

পুরোনো (value-based) মডেলে আমরা ভাবতাম `x`-এর ভেতরে সরাসরি `3` আছে। কিন্তু আসল (object-based) মডেলে ব্যাপারটা এমন:

- `x` ধরে রাখে একটা int object-এর **id**, আর সেই object-এর ভেতরে আছে `3`।
- `word` ধরে রাখে একটা str object-এর **id**, আর সেই object-এর ভেতরে আছে `'bonjour'`।

মানে `x` বাক্সের ভেতরে `3` নেই — সেখানে আছে এমন একটা object-এর ঠিকানা (id) যার মান `3`।

Expression evaluate করলে কী হয়

আগে বলা হতো "যেকোনো expression evaluate করলে একটা value পাওয়া যায়।" এখন আরও সঠিক করে বলা যায় — evaluate করলে একটা object-এর **id** পাওয়া যায়। কোন object, তা নির্ভর করে expression-এর ধরনের ওপর:

- **Literal** (যেমন `176.4` বা `'hello'`) হলে Python একটা নতুন object বানায়।
- **Variable** হলে Python সেটা খুঁজে দেখে। না থাকলে `NameError` দেয়; থাকলে সেই variable-এ রাখা id-টা ফেরত দেয়।
- **Binary operation** (যেমন `+`, `%`) হলে দুই পাশের operand আগে evaluate করে, তারপর result-এর জন্য একটা নতুন object বানিয়ে তার id ফেরত দেয়।

Assignment statement কীভাবে কাজ করে

`a = b` লিখলে যা হয়, ক্রম অনুযায়ী:

- ডান পাশের (right-hand side) expression আগে evaluate হয় → একটা object-এর id পাওয়া যায়।
- বাম পাশের variable না থাকলে নতুন করে বানানো হয়।
- ডান পাশ থেকে পাওয়া id-টা বাম পাশের variable-এ রাখা হয়।

Compound data (list, set, dict, data class)

এখানে আসল মজা। List, set, dictionary — এরা নিজেরা সরাসরি value রাখে না, বরং অন্য object-গুলোর id রাখে।

`lst = [1, 2, 3]` করলে আসলে **চারটা আলাদা object** তৈরি হয়:

- `1`, `2`, `3` — প্রতিটার জন্য আলাদা int object,
- আর list-টা নিজে একটা object, যেটা ওই তিনটা int object-এর id ধরে রাখে।

Dictionary-র ক্ষেত্রে `{'a': 1, 'b': 2}` করলে মোট **পাঁচটা** object তৈরি হয় (দুইটা key, দুইটা value, আর dict নিজে)।

Convention: যে object **immutable** (পরিবর্তন করা যায় না, যেমন int, str), তার চারপাশে **ডবল বাক্স** আঁকা হয় — বোঝানোর জন্য যে এর ভেতরে ঢুকে কিছু বদলানো যাবে না।

সবচেয়ে দরকারি অংশ: Reassignment vs Mutation

এই দুইটার পার্থক্য বোঝাটাই এই পুরো নোটের আসল লক্ষ্য।

Reassignment — variable-কে নতুন object দেখাতে বলা

```
s = [1, 2]
s = ['a', 'b']
```

এখানে `['a', 'b']` নামে একটা নতুন list object তৈরি হয়, আর `s` এখন নতুন object-এর id ধরে। পুরোনো `[1, 2]` object-টা **বদলায় না** — শুধু `s` এখন অন্য দিকে তাকায়। তাই id চেক করলে আগে-পরে id আলাদা:

```
s = [1, 2]
id(s)    # ধরো 1695325453760
s = ['a', 'b']
id(s)    # 1695325453248 - ভিন্ন id
```

Mutation — object-টাকেই ভেতর থেকে বদলে দেওয়া

```
s = [1, 2]
list.append(s, 300)
```

এখানে কোনো নতুন list বানানো হয় না (যদিও `300`-এর জন্য একটা নতুন int object বানে)। পুরোনো list object-টার ভেতরেই `300`-এর id যোগ হয়ে যায়। আকার বদলালেও list-এর **id একই থাকে**:

```
s = [1, 2]
id(s)    # 1695325453760
```

```
list.append(s, 300)
id(s) # 1695325453760 — একই id
```

মাঝামাঝি কেস — index-এ assign করা

```
s = [1, 2]
s[1] = 300
```

এটা একধরনের reassignment, কিন্তু variable-এর নয় — object-এর **ভেতরের একটা id** বদলে দেওয়া হয়। তাই এটা object-কে **mutate** করে, কিন্তু `s`-এর id একই থাকে।

বিষয়	Reassignment	Mutation
কী হয়	variable নতুন object দেখায়	পুরোনো object-ই ভেতর থেকে বদলায়
নতুন object	হ্যাঁ, নতুন object তৈরি হয়	না, একই object
<code>id()</code>	বদলে যায়	একই থাকে
উদাহরণ	<code>s = ['a', 'b']</code>	<code>s.append(300) / s[1]=300</code>

এক লাইনে সারমর্ম: Variable id রাখে, object data রাখে। **Reassignment** variable-কে নতুন object দেখায় (id বদলায়), আর **mutation** পুরোনো object-টাকেই ভেতর থেকে পাল্টায় (id একই থাকে)। `id()` function দিয়ে যাচাই করতে পারবে কোনটা ঘটল।

নোট: মূল কোর্স নোটের সরলীকৃত বাংলা সারসংক্ষেপ। সম্পূর্ণ বিবরণ ও diagram-এর জন্য CSC110/111 কোর্স নোট দেখো।